



TU DUBLIN, TALLAGHT CAMPUS

MSC DEVOPS

**A comparison of AKS and K3s running on
Azure Virtual Machine Scale Sets**

Craig Dillon

School of Enterprise Computing and Digital Transformation

Supervised by

Dr. Omar Portillo

School of Enterprise Computing and Digital Transformation

22 January 2025

Declaration

I hereby certify that the material, which I now submit for assessment on the programmes of study leading to the award of Master of Science, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution.

Craig Dillon
22 January 2025

Acknowledgements

I would like to express my heartfelt thanks to Omar Portillo for his invaluable guidance, expertise, and encouragement throughout the course of this research. His insightful feedback and constructive suggestions played a pivotal role in shaping the direction and quality of this work.

I am also deeply grateful to the lecturers of the MSc DevOps course at TU Dublin for presenting me with numerous challenges that significantly contributed to the foundation of this research. Their expertise and support were crucial in my academic development.

A special note of thanks goes to Kevin Dermody and HEAnet for their support and for encouraging me to pursue this course, helping me expand my skills and knowledge in ways I hadn't anticipated.

Finally, I cannot thank my family enough for their continued support and understanding during the long hours of research and writing.

List of Figures

1	K3s deployment workflow	22
2	Online Boutique service architecture, viewed using Hubble	24
3	Prometheus CPU usage query	26
4	Prometheus RAM usage query	27
5	Prometheus pod startup time query	27
6	Prometheus batches processed query	27
7	Comparison of idle resource usage for AKS, K3s and K3s excluding the control plane	30
8	Performance comparison of AKS and K3s, iterations are in thousands	31
9	Performance comparison of AKS and K3s, batches are scaled down by a factor of 10	33
10	Performance metrics comparison for AKS and K3s clusters simul- taneously running Boutique and TensorFlow workloads.	35
11	Efficiency comparison of AKS and K3s for iterations (Boutique) and batches (TensorFlow) per unit of CPU and RAM	36
12	Impact on performance when run simultaneously compared to individual tests	38
A.13	Python code for configuring TensorFlow training model	49
A.14	K6 test for Online Boutique	51
A.15	Terraform code to deploy AKS cluster	52
A.16	Command line to install Cilium on AKS	52
A.17	Terraform code for deploying VMSS	53
A.18	Bash script for deploying K3s on VMSS	57

List of Tables

1	Cluster setup comparison	23
2	Percentage differences for resource usage when idle between AKS, K3s and K3s excluding the control plane	29
3	Comparison of weekly costs of each cluster at idle and during testing	40

Contents

0.1	Glossary	7
1	Introduction	10
1.1	Purpose of this study	10
1.2	Scope of this study	11
1.3	Research questions	12
1.4	Significance of the study	12
2	Literature review	13
2.1	Kubernetes	13
2.2	Lightweight Kubernetes distributions	14
2.3	Conclusion	15
3	Background	15
3.1	Kubernetes	15
3.2	Overview of AKS	16
3.3	Overview of K3s	17
4	Methodology	18
4.1	Experiment design	18
4.1.1	Objective	18
4.1.2	Variables	18
4.2	Testing environment setup	20
4.2.1	Detailed setup of AKS	20
4.2.2	Detailed setup of K3s	21
4.2.3	Cluster setup comparison	23
4.3	Application setup and testing parameters	23
4.3.1	Online Boutique	23
4.3.2	TensorFlow	24
4.4	Deployment and monitoring tools	25
4.5	Understanding the metrics used	26
5	Results and analysis	28

5.1	Idle analysis	28
5.2	Performance analysis	31
5.2.1	Online Boutique	31
5.2.2	TensorFlow	32
5.2.3	Combination tests	34
5.3	Cost comparison	39
5.4	Operational complexity analysis	41
6	Discussion	42
6.1	Summary of key findings	43
6.2	Implication in real world application	44
6.3	Limitations of the study	46
6.3.1	Limited scope of testing	46
6.3.2	Limited testing conditions	46
6.3.3	Infrastructure variability	46
6.3.4	Limited performance metrics	47
6.3.5	User specific customisations	47
7	Conclusion	47
7.1	Future work	48
A	Appendices	49
A.1	Detailed testing configurations	49
A.2	Example code for environment setup	52

0.1 Glossary

AI Artificial Intelligence.

AKS Azure Kubernetes Service, a managed Kubernetes service offered by Microsoft Azure.

CLI Command Line Interface.

CNCF Cloud Native Computing Foundation, a Linux Foundation project focused on container technology.

CNI Container Network Interface, a Kubernetes plugin for managing the cluster network.

CPU Central Processing Unit.

gRPC A cross-platform Remote Procedure Call framework that enables communication between distributed systems.

HCL HashiCorp Configuration Language, a human readable language for defining infrastructure as code, used with tools like Terraform.

HPA Horizontal Pod Autoscaler, a Kubernetes controller that automatically adjusts the number of pods in a deployment based on defined triggers and metrics.

IaC Infrastructure as Code, use of configuration files to provide automated, repeatable infrastructure.

K3s A lightweight Kubernetes distribution.

ML Machine Learning.

PVC Persistent Volume Claim, a request for storage in Kubernetes for use by applications.

QoS Quality of Service.

RAM Random Access Memory.

SCSI Small Computer System Interface, a set of standards for connecting and transferring data between computers.

TiDB An open-source scalable SQL database.

UI User interface.

VMSS Virtual Machine Scale Set, an Azure service that can deploy a group of identical, automatically scaled virtual machines.

YAML YAML Ain't Markup Language, configuration language used with Kubernetes and other infrastructure-as-code tools.

Abstract

Kubernetes has become the dominant platform for container orchestration, offering a wide range of features and technologies for deploying and managing microservices. While vendor-managed Kubernetes services like AKS provide operational simplicity, self-managed lightweight distributions like K3s are an interesting alternative proposition.

This study presents a comprehensive comparison of AKS and K3s running on VMSS, focusing on performance, scalability, cost-effectiveness and operational complexity. Test environments were deployed using Terraform, tested under identical workloads and metrics were collected by Prometheus and Grafana.

The results highlight the trade-offs between a vendor-managed Kubernetes service and a lightweight, self-managed alternative. AKS offers operational simplicity and better scalability at the cost of customisation and access to the control plane. K3s performed with excellent resource efficiency and can be considered an efficient and cost-effective option, albeit in exchange for increased operational complexity.

1 Introduction

In recent years, cloud-native computing has seen large adoption and development, with containerisation and orchestration platforms playing a key role in providing scalable and efficient application deployment. According to the Cloud Native Computing Foundation (CNCF), Kubernetes is the industry leader with 66% of consumers surveyed using it in production, with a further 18% evaluating it, (CNCF 2024). This adoption and growth has spawned multiple different orchestration platforms based on Kubernetes, including lightweight platforms like K3s, Microk8s, MicroShift and k0s, (Koziolek and Eskandani 2023).

This paper will explore and compare two Kubernetes solutions, a feature-rich and vendor-managed service, Azure Kubernetes Service (AKS), and a self-managed lightweight platform, K3s. The primary objective of this research is to evaluate the performance, cost-effectiveness and operational complexity of both platforms in a controlled environment by focusing on two applications to replicate real-world workloads, Online Boutique, a microservices benchmark tool by Google and TensorFlow, an open-source machine learning (ML) platform. This research will provide insights into how each Kubernetes platform performs under load and will consider the compromises between cost, complexity and performance to highlight their suitability for different use cases.

1.1 Purpose of this study

The purpose of this study is to compare AKS and K3s running on a Virtual Machine Scale Set (VMSS), evaluating their suitability for production workloads. By examining key aspects such as performance, resource consumption, cost-effectiveness and deployment complexity, the research will establish the relative strengths and weaknesses of each platform. The goal is to provide organisations with data-driven insights that will aid in making an informed decision when choosing a Kubernetes based platform.

This comparison also features scenarios where one platform may outperform the other, providing a deeper understanding of each platform’s strengths and weaknesses for enterprises and developers. By performing intense testing simulating real-world workloads, the study will contribute to wider discussion on comparing Kubernetes deployments for cost, performance and operational trade-offs.

1.2 Scope of this study

The focus of this study is two Kubernetes platforms, AKS and K3s, selected based on their contrasting approaches to container orchestration. AKS is a vendor-managed service that simplifies deployment and operation at the expense of some customisation, where K3s is a lightweight alternative designed for resource constrained environments and offers a greater level of control in configuration and deployment. The study compares the distributions under identical, intense workloads.

The scope is limited to deployments on Azure platforms, AKS and VMSS, with the design of the K3s deployment focusing on replicating the architecture of AKS as closely as possible to ensure a like-for-like testing environment. The purpose of this study was to compare the platforms as closely as possible, therefore it must be acknowledged that different configurations, optimisations or deployment environments of either platform may yield different results. In standardising the environments, this study prioritised a consistent methodology in comparing the effectiveness of either platform.

1.3 Research questions

Key questions that this study addresses are:

- RQ1: Performance - How does each platform perform under different workloads, both individually and simultaneously?
- RQ2: Scalability - How does each platform respond to increases in demand, how effectively does it scale resources and what impact does this have on performance?
- RQ3: Cost Effectiveness - What are the financial implications of deploying these platforms, how does the cost relate to the relative performance?
- RQ4: Operational Complexity - What are the differences in deployment and management complexity between each platform?

1.4 Significance of the study

The significance of this study is in its contribution to the evolving landscape of cloud-native computing, focusing on Kubernetes. This research provides a structured comparison of two options, AKS and K3s, offering insights that may be valuable for organisations and teams who are deciding on a platform that suits their needs.

By providing empirical data on performance, cost-effectiveness and operational complexity, stakeholders are equipped with detailed reference points for understanding potential compromises when choosing AKS or K3s. The workloads employed in the testing represent diverse use cases, with relevance in both general application hosting and specialised tasks. Comparing the operational and management effort offers insights that can help make decisions based on the technical knowledge and availability of engineering resources when comparing different platforms.

The findings of this study provide a starting point for further research into Kubernetes distributions, including scenarios where specialised workloads are tested or where clusters are customised to achieve specific goals.

2 Literature review

Kubernetes has established itself as the leading platform for container orchestration in a range of environments, from cloud to edge-networks. It has shown to be adaptable and scalable which has enabled widespread adoption across industries. Due to the fact that Kubernetes is an open-source platform, a variety of distributions have been created to meet specific needs with lightweight versions gaining particular popularity. This section provides insights into the current state of research into Kubernetes and lightweight alternatives, highlighting key differences, challenges and opportunities.

2.1 Kubernetes

Kubernetes uses a declarative model where the desired configuration is defined in YAML manifests. The control plane continually reconciles the cluster state to maintain the desired state while also providing automated scaling and self-healing properties, (Mfula, Ylä-Jääski and Nurminen 2021).

Recent research has focused on optimising Kubernetes scaling and performance. For example, a model leveraging CI/CD pipelines to enhance microservice efficiency has been proposed, (Turin et al. 2023). The study highlights the challenges in isolating pods, which can often lead to resource contention and offers a solution based on predicting resource load under certain workloads by using granular sampling strategies.

Despite its strengths and widespread adoption, Kubernetes presents challenges as a result of its dynamic and distributed nature. Its ability to scale workloads dynamically can lead to complexities in ensuring data remains persistent and performance remains acceptable. A study into the growing adoption of Kubernetes for stateful database workloads in highly available environments found that relying on Kubernetes Persistent Volume Claims (PVC) could provide persistent storage for databases that met performance demands, (Sharma 2023). When combined with operators and Helm charts, certain database workloads would scale efficiently. However, there were trade-offs in resource consumption, and the operator of one of the tested databases, TiDB, faced limitations in scaling demands. This indicates that operators need continued refinement to fully leverage Kubernetes' capabilities.

2.2 Lightweight Kubernetes distributions

As Kubernetes has matured and found increasing adoption, developers have identified a need for lightweight distributions capable of operating in resource constrained environments. This has led to the development of distributions that can run on a wide range of hardware in a variety of environments. Distributions include K3s, Microk8s and Kubespray, (Mušić, Hribar and Fortuna 2024) that can run on local computers, Raspberry Pi clusters or dedicated Virtual Machines hosted in the cloud or on premises, (Heckmann and Ravindran 2023) (Lawrence 2024), (Schumann 2024). Another notable development is the creation of operating systems that are designed specifically for containerised workloads, such as Fedora CoreOS and Flatcar Linux, both of which adhere to CNCF principles, (Bryant et al. 2024).

A comparative study of Kubernetes, Microk8s and K3s, demonstrated that K3s can achieve comparable scaling performance compared to Kubernetes and outperforms Microk8s, (Telenyk et al. 2021). The findings showed that K3s required the least amount of resources in an idle state but relied on the highest CPU load when performing certain node scaling actions. Research examining comparable metrics between these distributions found that K3s performed similarly to Kubernetes in node scaling performance, with both outperforming MicroK8s, (Bohm and Wirtz 2021).

While K3s excels in small deployments, additional engineering effort is required to replicate the scale of larger Kubernetes clusters, (Farrel et al. 2023). Doing so requires consideration on how to manage the cluster as nodes require a secret token to join, (Todorov 2022) and default modules bundled with K3s should be evaluated for their ability to scale to the desired size, (Perez et al. 2023), (Farrel et al. 2023).

2.3 Conclusion

The current state of research exemplifies Kubernetes’ versatility and the growing development and adoption of lightweight distributions. However, these studies often rely on minimal or synthetic testing environments and in the case of lightweight distributions, focus on resource limited hardware.

By comparing AKS and K3s on VMSS under conditions that replicate realistic production clusters, this study focuses on real-world usability. The findings aim to provide actionable insights for organisations making the transition to Kubernetes or seeking to optimise their existing Kubernetes deployments.

3 Background

3.1 Kubernetes

Kubernetes is an open-source orchestration platform for deploying, scaling and managing containerised microservices. It offers high availability by organising and scheduling containers across a cluster of nodes, providing scalability under demand.

Typical Kubernetes architecture consists of at least one control plane and one worker node. The control plane includes components like the API server, scheduler, controller manager and oversees the overall state and operation of the cluster. It receives and processes commands, manages state and monitors health. Worker nodes run the application workloads within containers orchestrated by pods, the smallest deployable Kubernetes unit, (Darbinyan 2024).

Some key components of Kubernetes are as follows:

- Pods: The basic unit, typically containing one or more containers that share resources like networking and storage
- Services: Endpoints for network communication, allowing pods to interact reliably despite their ephemeral nature
- Deployments: Declarative objects that manage the lifecycle of pods, handling replication, updates and rollbacks
- Namespaces: Logical partitions within a cluster used to isolate resources and manage permissions

(Turin et al. 2023).

3.2 Overview of AKS

AKS is a vendor-managed Kubernetes solution provided by Microsoft, offering managed backend services like virtual networks, control planes and other Azure native components. AKS integrates with many Azure services including Azure Active Directory which can provide Role Based Access Control, Azure Monitor and support for Azure DevOps pipelines which streamlines containerised deployments, (Franssens, Gopalakrishnan and Lenz 2021).

AKS has shown that it is capable of handling diverse and intense loads due to its ability to scale vertically and horizontally in a dynamic manner. This ensures high resource availability, especially in scenarios with unpredictable traffic patterns, (Normelli 2024),(Sharma 2023). Vendor-managed Kubernetes services is a competitive market and AKS does find itself at a disadvantage compared to its competitors when comparing maximum cluster sizes and pricing, (Kumar et al. 2024).

To summarise, AKS is a well established, enterprise ready managed Kubernetes solution that is suited for organisations that favour integration, scalability and ease of use. A managed control plane and integration with core Azure services further solidifies it as an option for organisations who are adopting container orchestration with a focus on minimising setup and management complexity.

3.3 Overview of K3s

K3s is a lightweight Kubernetes distribution developed by Rancher Labs and designed to run Kubernetes in resource-constrained environments, (Telenyk et al. 2021). A key feature of K3s is its simplified architecture where components of Kubernetes that may be considered surplus are omitted. This reduces resource overhead, allowing it to run in environments with limited resources, (Bahy et al. 2023).

Despite stripping out components to create a lightweight package, K3s is Kubernetes compliant and a CNCF certified platform, (Telenyk et al. 2021). K3s offers a flexibility that accommodates a wide range of customisation that can be deployed in diverse environments supporting multiple computing architectures, from cloud services and on premises, to remote military networks or Raspberry Pi clusters, (Sorensen 2023),(Bohm and Wirtz 2021),(Fogli et al. 2021).

K3s offers an alternative to a feature-rich Kubernetes distribution, offering a lightweight, highly customisable and efficient solution for microservices deployments. Its simplicity and resource efficiency make it a valuable proposition for teams who may be working in resource constrained environments or desire a platform that can be customised to meet their needs. However, this comes at a compromise as deployments can quickly become complex and careful consideration must be applied to ensure the engineering demand can be met.

4 Methodology

4.1 Experiment design

The goal of this study is to evaluate and compare the performance, scalability and operational efficiency of AKS and a K3s cluster deployed in an identically sized VMSS.

The comparison was performed in an environment designed to minimise the impact of variables under the control of the cloud provider and to focus on the differences between the two cluster types. Both AKS and VMSS are subject to underlying cloud provider configurations, some which cannot be modified by the customer. Controlled variables were implemented, and the testing environment was designed, to ensure differences between the cluster types was minimal and to ensure consistency and reliability in the comparison.

4.1.1 Objective

This study aims to assess the ability of both clusters to handle a variety of workloads, relying on metrics such as resource utilisation, scaling behaviours and operational overhead. This can be broken down into:

- Performance: A comparison of cluster CPU and RAM resource usage and application performance. [RQ1]
- Scalability: The ability of each cluster to scale pods and nodes in response to stress testing different workloads. [RQ2]
- Cost: A comparison of financial cost of running each cluster during the testing period. [RQ3]
- Operational Complexity: A comparison of the effort in deploying and managing each cluster. [RQ4]

4.1.2 Variables

The experiment involves a range of variables that need to be considered. Controlled variables, independent variables and dependent variables should be identified.

4.1.2.1 Controlled variables

To ensure a fair comparison, the following elements were controlled on both environments:

- Virtual Machine Specifications: Both clusters use Standard_D2as_V5 virtual machines with 2 CPU, 8 GB RAM and 75 GB SCSI storage.
- Node Count: AKS was configured with a minimum of 1 node and maximum of 10 nodes. K3s was configured with a minimum of 2 nodes and maximum of 11 nodes, to account for the first K3s node performing the role of control plane.
- Kubernetes Version: Both clusters used Kubernetes version 1.28.5.
- Operating System: Ubuntu 22.04 LTS is the base OS for both clusters.
- Network Plugin: Both clusters used Cilium as the network plugin with Hubble enabled, shared load balancer mode and Gateway API as the ingress method.
- Value Store: Kubernetes relies on a key-store to manage state across the cluster. Etcd is deployed as standard with AKS and cannot be changed. Therefore, K3s was deployed with etcd instead of the default Embedded SQLite backend.
- Workloads: Identical applications were deployed on both clusters to simulate real world usage and to stress test CPU and RAM usage.
- Monitoring Tools: Prometheus and Grafana (in the form of the Promstack deployment) were used for metric collection and visualisation.
- Testing Scenarios and Duration: Identically specified tests were performed at the exact same time to account for the possible impact of co-tenants.

Kubernetes Gateway API was chosen over the older Kubernetes Ingress mechanism due to issues arising around QoS and service interruptions, (Mondal, Zheng and Cheng 2024). While there are several options available for ingress control, Gateway API is the next generation Kubernetes ingress which can handle both north/south and east/west traffic, (Kuikka 2024).

Cilium, a CNI (Container Network Interface), was chosen for its ability to provide both a network layer and service mesh for the clusters without creating unnecessary overhead, as there is no need for sidecar pods which is common with other service meshes, (Karakacs et al. 2023).

4.1.2.2 Independent & dependent variables

The independent variables in this experiment are the cluster types, AKS and K3s running on VMSS. The independent variables directly impact the dependent variables, which are performance, scalability, cost-effectiveness and operational complexity.

Other variables for consideration are environmental factors such as cloud provider optimisations or pre-bundled software. All measured metrics include standard deviations to take into account variability in cloud provider resource availability.

4.2 Testing environment setup

Each cluster was deployed using Terraform to provide infrastructure that is uniform, consistent and requires no manual intervention.

4.2.1 Detailed setup of AKS

The AKS cluster was deployed with the following configurations:

4.2.1.1 Node configuration

A minimum of 1 node and maximum of 10 nodes with default autoscaling enabled. The node size used was Standard_D2as_V5. The nodes are running Ubuntu 22.04 LTS.

4.2.1.2 Cluster configuration

The cluster was created with Kubernetes version 1.28.5, a dedicated Azure Virtual Network and Load Balancer, with Cilium as the Kubernetes network plugin. Kubernetes Gateway API is the ingress method and used in tandem with Cert-Manager for SSL termination, relying on the DNS challenge method.

4.2.1.3 Deployment workflow

AKS was deployed using Terraform, specifically disabling the Kubernetes networking plugin to allow Cilium to be installed in its place.

4.2.2 Detailed setup of K3s

K3s was deployed using Terraform and used a post deployment bash script to install K3s and other components. This is a hard requirement as VMSS are created as standard Virtual Machines and require a firstboot script to ensure new nodes create the desired environment when starting up. A storage blob was created beforehand to store deployment scripts and K3s cluster keys. This allows the primary K3s node to upload the required information for subsequent nodes to join the cluster when starting up.

4.2.2.1 Node configuration

The K3s nodes were created using the Standard_D2as_V5 Virtual Machine size with autoscaling set to automatic using the default configuration. There are a minimum of 2 nodes, to account for the first node performing the duty of the control plane and a single worker node. The maximum amount of nodes is set to 11 and the control plane has been tainted to avoid applications being installed on it. The base operating system is Ubuntu 22.04 LTS.

4.2.2.2 Cluster configuration

This cluster is running K3s, built on Kubernetes version 1.28.5 with a dedicated Azure Virtual Network, an Azure Load Balancer and Cilium as the Kubernetes network plugin. Kubernetes Gateway API is the ingress method and Cert-Manager is performing SSL termination using DNS challenge for certificate creation and verification.

4.2.2.3 Deployment workflow

To deploy K3s on Azure VMSS and enable dynamic creation and removal of nodes requires several steps:

- Terraform: Used to deploy the VMSS, Virtual Network, Load Balancer and other Azure network configurations. Terraform also deploys the cloud-init payload.
- Cloud-init: Once a VMSS node starts up, cloud-init collects the K3s install bash script from the Azure storage account and runs it.
- K3s install script: A bash script that will either install the control plane based on the hostname (K3sVMSS000000) or install K3s as a worker node and join the cluster. This script also installs and configures Cilium on the control plane.

See Figure 1 for a diagram outlining this process.

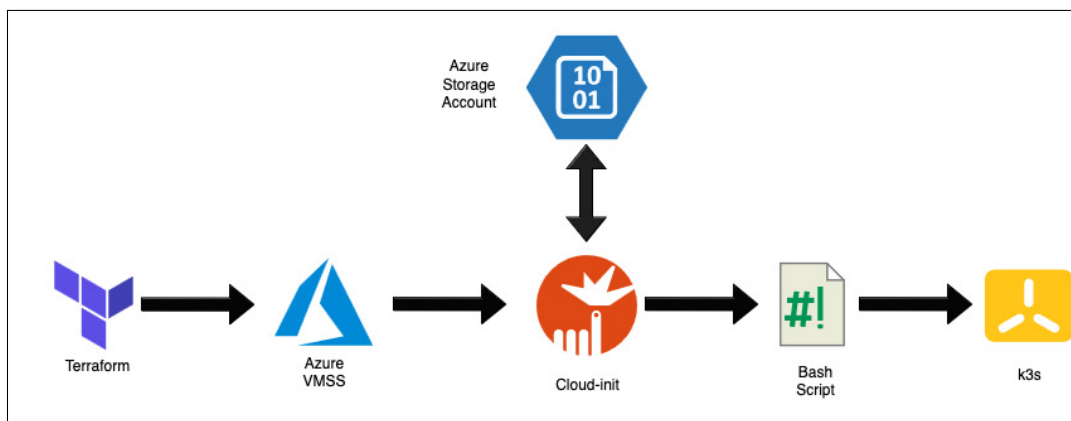


Figure 1: K3s deployment workflow

4.2.3 Cluster setup comparison

Table 1 outlines the main features of each deployment, with the control plane and autoscaling as the only major differences.

Feature	AKS	K3s
Control Plane	Managed by Azure (no direct access)	Hosted on a user-managed VM
Node Count	1-10	2-11 (1 dedicated control plane)
Autoscaling	Enabled (AKS default autoscaler)	Enabled (default VMSS autoscaler)
Networking Plugin	Cilium	Cilium
Metrics	Prometheus & Grafana (Promstack)	Prometheus & Grafana (Promstack)
Provisioning Method	Terraform	Terraform, cloud-init & bash
Cluster Backend	etcd	etcd

Table 1: Cluster setup comparison

4.3 Application setup and testing parameters

Google’s Online Boutique microservices demo, and TensorFlow machine learning workloads were selected to stress test the clusters. By combining these two distinct workloads the testing methodology captures comprehensive insights into a broad scope of production-like scenarios, ensuring the evaluation reflects the diverse demands placed on Kubernetes clusters in high demand, real-world scenarios.

4.3.1 Online Boutique

Online Boutique is a fully functional demo application created by Google to demonstrate cloud-first microservices architecture. It features several languages and components that communicate using gRPC. Languages used are, Go, C#, Node.js, Python and Java, with Redis used for caching, (Google 2024). This workload is suitable for testing Kubernetes clusters as it relies on key features such as service discovery, load balancing, resource scaling and inter-service communication, see Figure 2 for a visual representation of the application interconnectivity.

Online Boutique simulates realistic scenarios, providing a functional UI where users can browse products, add them to their basket and checkout. These interactions simulate a real-world microservices application and provide a detailed evaluation of cluster performance and operation under load.

Online Boutique tests were performed with k6, an application created by Grafana Labs which was chosen to generate load and test the performance of the application and Kubernetes cluster due to its ability to perform reliably using minimal resources, compared to other tools like Taurus or Locust, (Ernerstedt 2023). The application defaults for Horizontal Pod Autoscaling (HPA) were used.

The test simulated 1000 users accessing the site, browsing for an item, adding it to the basket, completing the purchase then waiting for 1 second before repeating the process, see Figure A.14 for the full test script. The test ran for 30 minutes, against both clusters at the same time starting from an idle state. This was repeated 5 times.

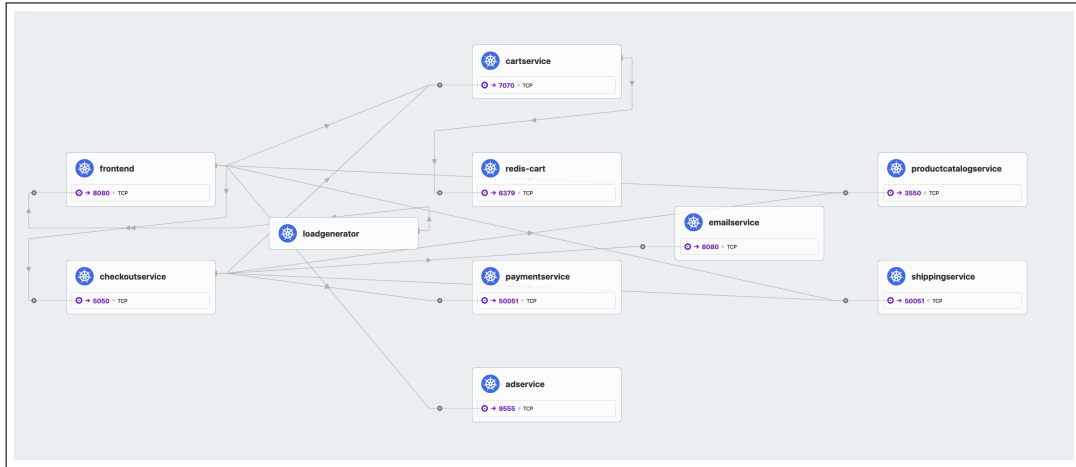


Figure 2: Online Boutique service architecture, viewed using Hubble

4.3.2 TensorFlow

TensorFlow is a Machine Learning platform that supports pod scaling in Kubernetes workloads and was included in testing to feature a computationally intensive task that challenges a clusters ability to handle high CPU and RAM demands.

Machine learning workloads are increasingly common in Kubernetes clusters, with a range of research focused solely on AI/ML in edge computing, (Koukis et al. 2024), (Cłapa, Grudzień and Sierszeń 2023).

The tests were performed using the Python based PyPI ecosystem, (Li, Rajbahadur and Bezemer 2024). The test used the CIFAR-10 dataset which is a collection of images in 32x32 pixel size. TensorFlow then resizes these images to 224x224 and then compares the accuracy against the validation set bundled with the CIFAR-10 dataset, (Chen et al. 2024). MobileNetV2 is a compact and efficient model that can operate at a reduced computational cost and has been pre-trained for image recognition to speed up the training process, (Gurin et al. 2024). The TensorFlow Prometheus plugin is included as part of the job as a means to extract the metrics accuracy, loss and batches completed. While accuracy and loss have no direct correlation to performance they offer insights to the stability and consistency of testing, (Nelli 2023). HPA was applied to this test with a minimum of 1 pod to a maximum of 10 pods, scaling up when total pod CPU utilisation averages 50% or more over 1 minute.

4.4 Deployment and monitoring tools

The deployment and management of the Kubernetes clusters required a combination of automation tools, scripting languages and monitoring tools to ensure reproducibility and in-depth insights.

Terraform was chosen for infrastructure deployment. It is a widely used Infrastructure as Code (IaC) tool, (Mehdi and Walia 2023), that is modular, supports multiple cloud and on premises platforms and uses HCL to declare desired infrastructure states, (Paloviita 2022).

Helm charts were used to deploy applications on the cluster. They can be described as an install package for Kubernetes that contains all required resources to deploy an application on a Kubernetes cluster, are written in YAML and can allow for customisation of the application, (Zerouali, Opdebeeck and De Roover 2023).

Cloud-init and Bash scripting were used as part of the VMSS deployment. Cloud-init provides contextualisation for the VMs upon startup by running OS configuration tasks, (Nagy et al. 2021) and a Bash script was used to install and configure K3s, depending on whether it was running on the control plane or worker node.

Hubble is a monitoring tool bundled with the Cilium CNI and provides a web UI for visualisation of application and namespace dataflows, (Zavarella 2022). Hubble was used to visualise application interconnectivity on the cluster and for troubleshooting network issues when designing the testing environment.

4.5 Understanding the metrics used

While the metrics chosen provide comprehensive insights to answer the research questions of this study, to have a true understanding of the results it must be clarified what is actually being measured.

CPU Usage

CPU usage is the primary indicator of compute resources. When monitoring CPU usage in Kubernetes with Prometheus it is calculated based on CPU time spent performing tasks across all available cores on the node or cluster, (Pivotto and Brazil 2023), (Holopainen 2021). See Figure 3 for the query used.

1 `sum(rate(container_cpu_usage_seconds_total{namespace="$namespace",container!="",pod!=""}[1m]))`

Figure 3: Prometheus CPU usage query

This calculates the average CPU time spent on active tasks, per minute, per namespace and the `container!="", pod!=""` commands exclude jobs related to infrastructure containers.

Memory Usage

Calculating memory usage is based on average physical RAM usage by pods in a specific namespace, again omitting jobs related to infrastructure containers. See Figure 4 for an example.

```
1 sum(container_memory_usage_bytes{job="kubelet", container!="", namespace="$namespace"})
```

Figure 4: Prometheus RAM usage query

Average Pod Startup Time

Pod startup time was calculated by creating an average of when a pod was scheduled by Kubernetes and when the pod passed all required healthchecks, entering a ready state, see Figure 5.

```
1 min_over_time(kube_pod_status_ready_time{job="kube-state-metrics", namespace="$namespace"}[1h])  
2 -  
3 min_over_time(kube_pod_status_scheduled_time{job="kube-state-metrics", namespace="$namespace"}[1h])
```

Figure 5: Prometheus pod startup time query

Batches processed

Capturing batches processed by TensorFlow involves creating a Prometheus endpoint and definition as part of the TensorFlow job, as seen on lines 7-13 in Figure A.13. This can then be queried using Prometheus using Figure 6.

```
1 sum(max_over_time(tensorflow_batches_processed{namespace="tensorflow\"}[1h]))
```

Figure 6: Prometheus batches processed query

This tracks the total amount of batches completed by TensorFlow training run.

Iterations completed

K6s provides an output for total iterations completed and checks for each stage of the test. The following are the stages of the complete test and a check is performed at the end of each step, where K6s will create a percentage pass rate of all total checks:

- Product page loaded
- Cart page loaded
- Order placed
- Checkout successful

Figure A.14 provides code for this test.

5 Results and analysis

Each test application operates within its own dedicated namespace, serving as the level at which measurements are taken. This approach captures additional overheads that may not be apparent to the end user and provides a comprehensive view of the resource consumption associated with the application.

5.1 Idle analysis

Before comparing the performance of the clusters under load, it is important to gain some insights into how the platforms behave when idle. The measurements were taken with both clusters configured identically, running the same applications, namespaces and deployments.

At idle, AKS creates 4 worker nodes to meet its minimum resource threshold. This configuration ensures redundancy and capacity to handle sudden surges in demand. However, this approach results in higher resource allocation while idle requiring a larger footprint compared to K3s. Table 2 illustrates that AKS has an average idle CPU usage of 0.75 compared to K3s at 0.4, a 46.6% difference and a 62.6% difference when compared to K3s excluding the Control Plane. Additionally, AKS shows higher memory usage at idle, consuming 8.3 GB RAM compared to 5.9 GB on K3s and 3.69 GB on K3s excluding the control plane, a difference of 28.9% and 55.5%, respectively.

Metric	AKS	K3s	K3s w/o CP	Unit
CPU Usage	0.75	0.4	0.28	CPU
Memory Usage	8.3	5.9	3.69	GB
% Difference				
Idle CPU	0	46.6	62.6	%
Idle Memory	0	28.9	55.5	%

Table 2: Percentage differences for resource usage when idle between AKS, K3s and K3s excluding the control plane

In contrast, K3s on VMSS maintains a single worker node, with a total of 2 nodes when accounting for the manually configured control plane. As a result of operating with fewer nodes, K3s uses a higher percentage of available resources. Figure 7 shows that K3s operates at idle using 10.1% of available CPU and 44.4% of available RAM. Further, K3s excluding the control plane operates with 14.38% of available CPU and 46.5% of available RAM, in comparison to AKS' usage of 9.34% of available CPU and 26.8% of available RAM.

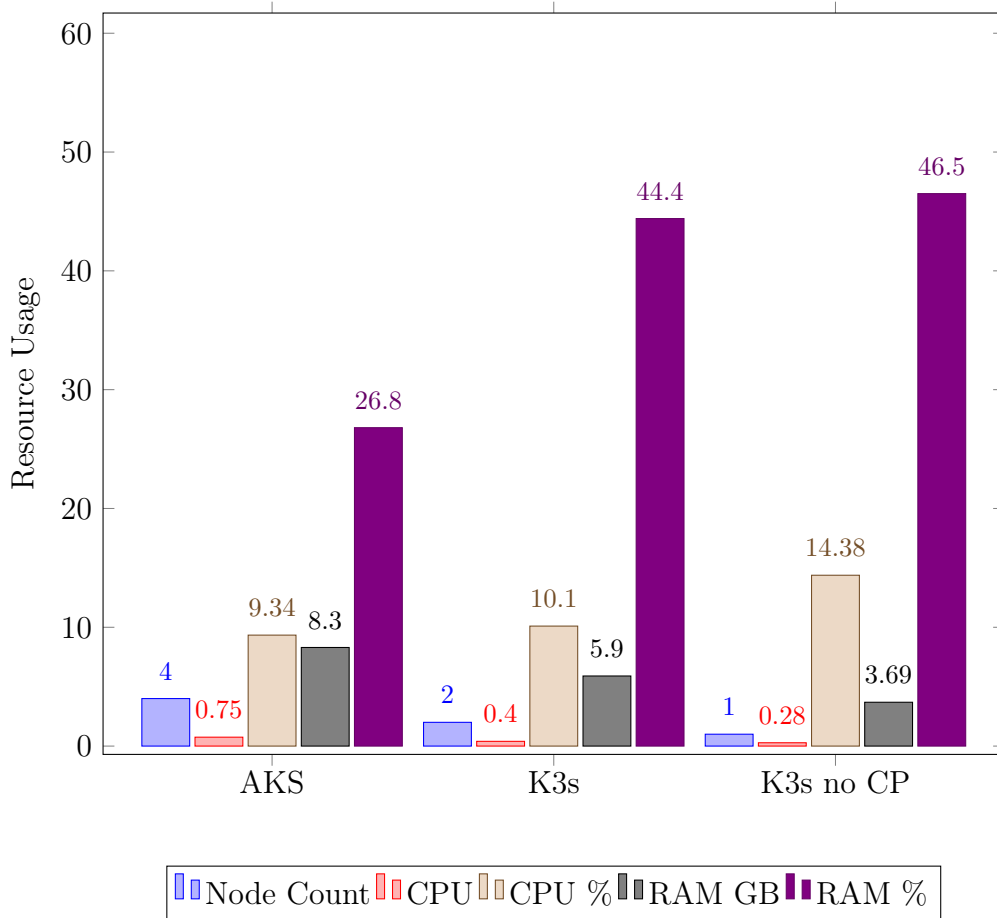


Figure 7: Comparison of idle resource usage for AKS, K3s and K3s excluding the control plane

5.2 Performance analysis

The performance analysis answers research question 1, "How does each platform perform under different workloads, both individually and simultaneously?" and research question 2, "How does each platform respond to increases in demand, how effectively does it scale resources and what impact does this have on performance?".

5.2.1 Online Boutique

In this section, a detailed comparison of AKS and K3s running Online Boutique, under load, is presented. The benchmarks measured are CPU usage, RAM (GB) usage, node count, pod count, pod startup time (in seconds), iterations of the test completed (in thousands) and the error rate. By examining these factors, the aim is to provide insights into performance, efficiency and consistency. All measurements are based on data from the allocated namespace for comparison.

As illustrated in Figure 8, AKS exhibits a significantly higher throughput, completing on average 35.4 thousand iterations compared to K3s, which, on average, completed 13.6 thousand iterations. The standard deviation data reveals an important contrast in that AKS also shows a much larger variation in iterations completed, while K3s maintains a more consistent iteration count, albeit at a lower total throughput.

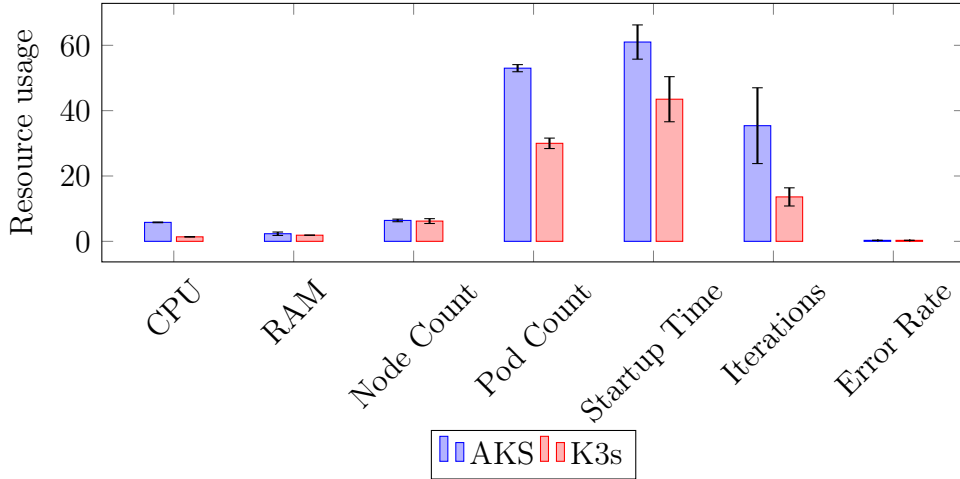


Figure 8: Performance comparison of AKS and K3s, iterations are in thousands

A deeper look at the other performance metrics in Figure 8 may provide clarity on the underlying reasons for these differences. AKS consumes significantly more CPU under load at 5.8 CPU compared to K3s at 1.38 CPU. AKS has slightly higher RAM usage, 2.3 GB, where K3s consumes 1.88 GB. AKS creates 53 pods compared to K3s 30 pods. These results reflect AKS' scalability and ability to allocate more resources under load. However, this increased resource allocation comes at the cost of consistency, as the fluctuations in performance become more noticeable under increased load. Pod startup times are another important factor to consider, with AKS taking an average of 61 seconds to start up pods compared to K3s, which starts pods in 43.5 seconds, on average.

Overall, this test suggests that AKS excels in throughput and scalability, it comes at the cost of higher resource consumption, longer pod startup times and larger variability in performance. K3s, in comparison, offers more efficient resource usage, faster pod startup times and more consistent overall performance.

5.2.2 TensorFlow

This section focuses on a comparative analysis of AKS and K3s while running a stress test on TensorFlow, focusing on metrics such as resource consumption, node and pod amounts, pod startup time and completed batches.

The results in Figure 9 demonstrate that K3s excels in this particular workload, completing on average 372.0 batches compared to AKS's average of 266.6. Further to this, K3s shows superior throughput in this workload despite its relatively lower resource allocation. AKS, in comparison, provisioned more nodes for the task, 10 nodes compared to K3s 6 nodes, yet does not achieve comparable throughput, highlighting that higher resource allocation does not always equate to better performance.

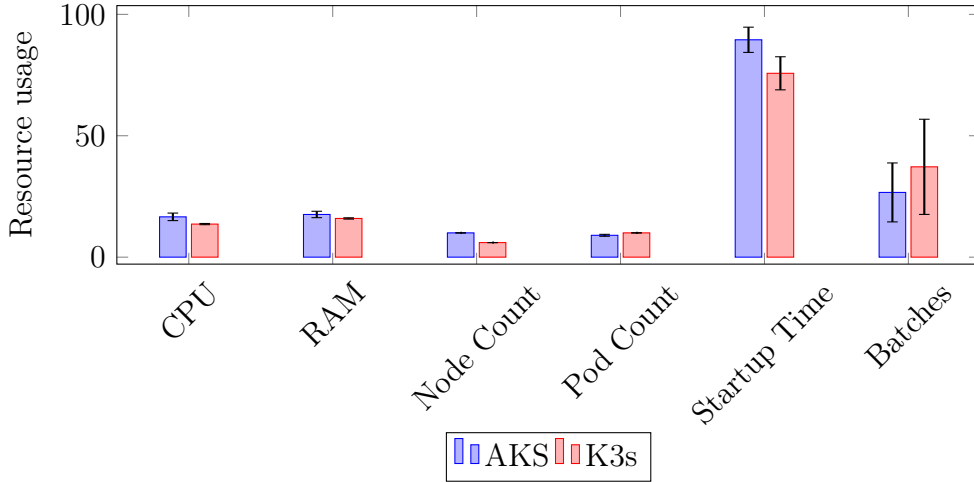


Figure 9: Performance comparison of AKS and K3s, batches are scaled down by a factor of 10

AKS averages 16.6 CPU and 17.58 GB of RAM usage in these tests, where K3s consumed 13.62 CPU and 15.94 GB of RAM on average (Figure 9). In terms of infrastructure, K3s was able to create more pods, 10 compared to 9 for AKS, on fewer nodes with a faster pod startup time. Overall, this suggests that K3s has a significant advantage with a high CPU, high RAM workload in that it can outperform AKS while creating fewer nodes and utilising fewer resources.

In conclusion, K3s is superior under TensorFlow workloads with its ability to complete more batches, with fewer nodes, lower resource usage and faster pod scaling abilities. There is one caveat, and that is consistency in batch sizes where K3s shows larger variances in comparison to AKS. AKS provides a more consistent, but smaller output at the expense of efficiency.

5.2.3 Combination tests

When running the tests simultaneously, the performance trends in the solo tests continue. AKS allocates more resources, deploying more pods to handle the combined workload which leads to higher resource usage and longer pod startup times. This approach continues to provide higher throughput for a lighter workload, as observed in the Online Boutique metrics. In comparison, K3s maintains a resource-efficient profile, continuing to outperform AKS in the TensorFlow workloads and matching or achieving lower pod startup times.

Figure 10 presents a comparative analysis of key performance metrics for AKS and K3s clusters running the Online Boutique and TensorFlow workloads simultaneously. For the Online Boutique workload, K3s outperforms AKS in terms of CPU usage (K3s 1.5 CPU vs. AKS 4.6 CPU) and pod startup time (K3s 49 seconds vs. AKS 66 seconds), with slightly lower RAM consumption (K3s 2 GB vs. AKS 2.3 GB) and a comparable pod count (K3s 31 pods vs. AKS 35 pods) which results in AKS averaging higher iterations complete (35,000) compared to K3s (12,900).

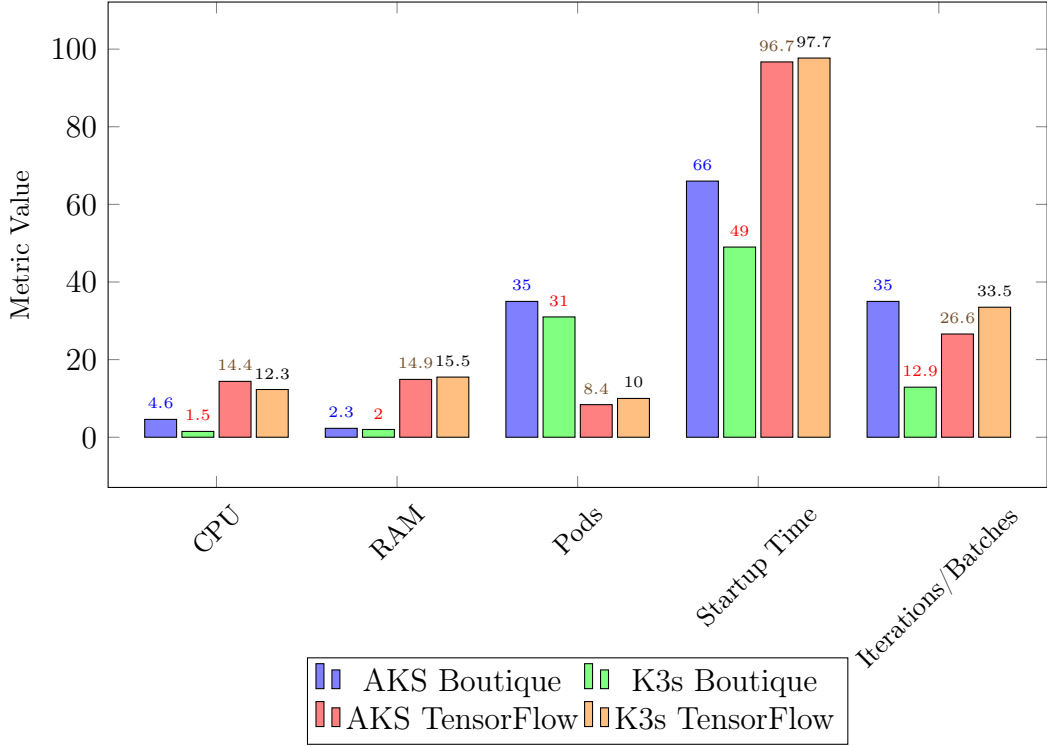


Figure 10: Performance metrics comparison for AKS and K3s clusters simultaneously running Boutique and TensorFlow workloads.

In TensorFlow workloads, K3s demonstrates lower CPU usage, (K3s 12.3 CPU vs. AKS 14.4 CPU) while consuming more RAM (K3s 15.5 GB vs. AKS 14.9 GB), achieving a higher average pod count (K3s 10 pods vs. AKS 8.4 pods) and completing more batches (K3s 335 batches vs. AKS 266 batches). TensorFlow is a memory intensive workload and K3s shows an ability to allocate RAM more effectively than AKS in this test.

These results highlight differences between the two clusters, with K3s being more resource-efficient for Online Boutique workloads and leveraging higher RAM usage to enhance TensorFlow performance. In comparison AKS outperforms K3s in iterations for Online Boutique, indicating its advantage in effectively allocating resources for multi-component microservices.

Additionally, K3s averaged scaling to 6 nodes compared to AKS reaching its maximum of 10 nodes in these tests. AKS's node scaling behaviour suggests an aggressive scaling policy, maximising resource allocation to support workloads to the point of possibly over-committing nodes. K3s comparatively appears more conservative, requiring fewer nodes which could be due to its lightweight, efficiency driven design.

Figure 11 illustrates the resource efficiency for the AKS and K3s clusters, measured as iterations completed per unit of CPU and RAM for the Online Boutique workload and batches completed per unit of CPU and RAM for the TensorFlow workload.

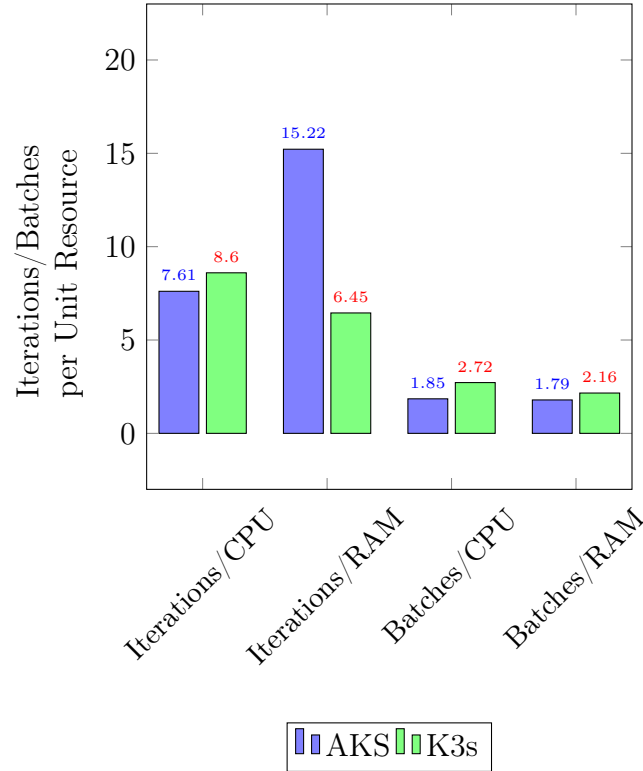


Figure 11: Efficiency comparison of AKS and K3s for iterations (Boutique) and batches (TensorFlow) per unit of CPU and RAM

For the Online Boutique workload, K3s achieves greater CPU usage efficiency with 8.6 iterations per CPU compared to AKS's 7.61 iterations per CPU. However, AKS outperforms K3s in iterations completed per GB of RAM, with 15.22 iterations per GB of RAM consumed compared to K3s with 6.45 iterations per GB RAM.

In the TensorFlow workload, K3s exhibits higher efficiency in both metrics, completing 2.72 batches per CPU and 2.16 batches per GB RAM, compared to 1.85 batches per CPU and 1.79 batches per RAM for AKS. These results suggest that K3s is better suited for workloads that rely on intensive batch processing, whereas AKS offers a more balanced resource utilisation profile better suited to multi-component microservice applications like Online Boutique.

Figure 12 shows the impact on performance when running simultaneous tests compared to individual tests. K3s experienced a 5.13% drop in completed iterations for the Online Boutique workload and a 10% decrease in completed batches for the TensorFlow workload when both tests were run simultaneously. On the other hand, AKS experienced a 1.35% drop in iterations for Online Boutique and saw no change in batches completed for TensorFlow compared to individual test results.

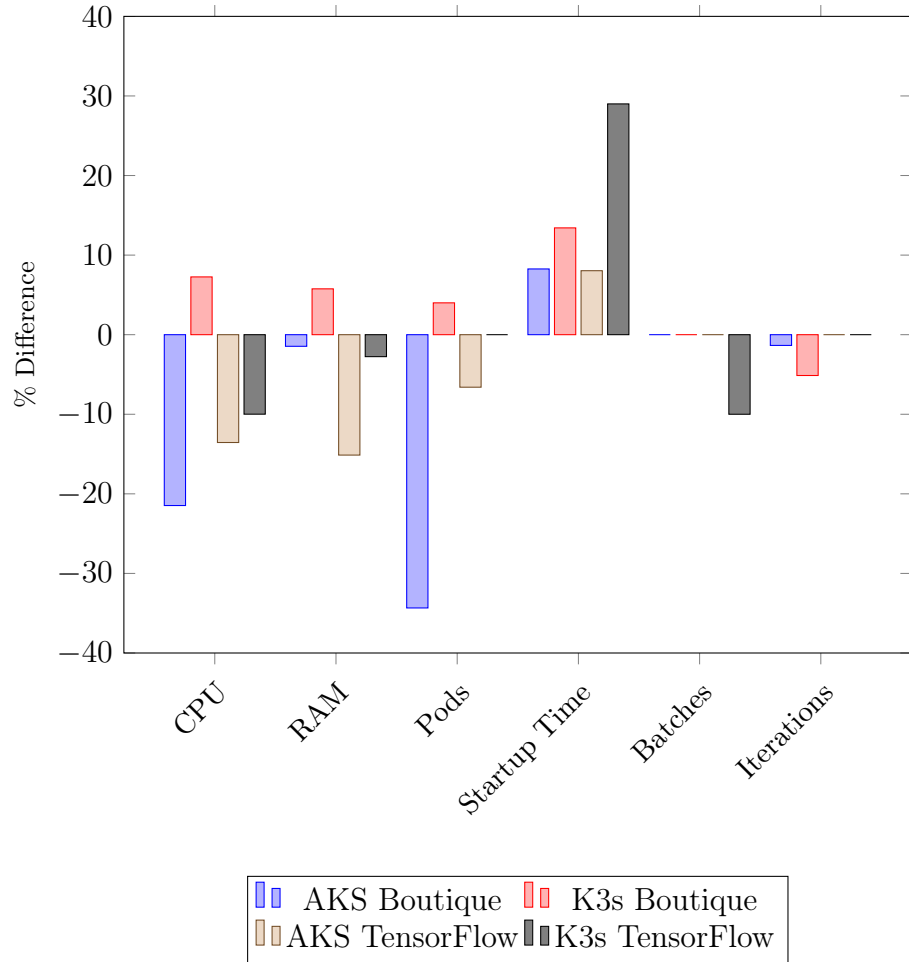


Figure 12: Impact on performance when run simultaneously compared to individual tests

Compared to individual workloads, simultaneous workloads show a distinct effect on resource allocation. AKS experienced a decrease in CPU allocation for Online Boutique (-21.47%) and TensorFlow (-13.55%) while K3s allocated more CPU resources for Online Boutique (7.26%) and a decrease for TensorFlow (-9.99%). RAM allocation paints a similar picture where AKS sees a decrease for Online Boutique (-1.46%) and TensorFlow (-15.13%), where K3s increases RAM allocation for Online Boutique (5.76%) and a slight decrease for TensorFlow (-2.76%).

Pod startup time increases compared to individual tests on all workloads, with AKS experiencing an increase of 8.26% for Online Boutique and 8.04% for TensorFlow. In comparison, K3s experiences greater increases in pod startup time, 13.42% for Online Boutique and 29% for TensorFlow in simultaneous testing. The amount of pods created per application was also impacted with AKS creating 34.34% fewer pods for Online Boutique and 6.6% less for TensorFlow. K3s created on average 4% more pods for Online Boutique and there was no change for TensorFlow in comparison to individual testing.

These findings suggest that AKS is better suited to scenarios requiring aggressive, robust scalability and minimal performance degradation. K3s, in comparison, may be better suited to situations where aggressive scaling is not a priority and efficiency, resource utilisation and cost-effectiveness has a higher priority over raw performance.

5.3 Cost comparison

The following section addresses research question 3, "What are the financial implications of deploying these platforms, how does the cost relate to the relative performance?". Table 3 provides a comprehensive breakdown of weekly costs, while idle and under load, for every Azure resource created for each cluster.

Under idle conditions, all components related to running AKS incur a weekly cost of €83.83, compared to K3s on VMSS which costs 29.3% less at €59.32. This is primarily driven by lower VM and storage costs, reflecting K3s smaller footprint. During workload testing, this difference narrows as K3s incurs a weekly cost of €77.10 compared to €91.01 for AKS under the same conditions. The increase in K3s costs reflects the clusters scaled up resource usage, particularly with Virtual Machines at €45.57 and Microsoft Defender costs at €21.32. However, K3s remains more cost-effective than AKS, running at 29.2% lower cost in idle conditions and 15.3% less under load.

Resource	AKS idle	AKS Load	K3s idle	K3s load
VM	€47.57	€51.31	€35.28	€45.57
Storage	€14.23	€14.63	€0.7	€3.07
MS Defender	€3.61	€3.71	€18.20	€21.32
Load Balancer	€6.37	€8.12	€4.44	€6.37
Virtual Network	€2.66	€3.07	€0.70	€0.77
DNS	€0.10	€0.10	€0.00	€0.00
Cluster Management	€9.36	€10.08	€0.00	€0.00
Total	€83.83	€91.01	€59.32	€77.1

Table 3: Comparison of weekly costs of each cluster at idle and during testing

A snapshot of a single day costing, when running combination tests, shows that AKS cost €13.43 compared to K3s on VMSS at €8.53. During this period, AKS cost 57.4% more than K3s but showed more stable performance overall. This can be further broken down into a comparison of the financial cost of performance degradation when running simultaneous workloads as follows:

Online Boutique

- AKS: Cost per 1% performance drop = €9.95 (€13.43/1.35)
- K3s: Cost per 1% performance drop = €1.66 (€8.53/5.13)

TensorFlow

- AKS: No drop in performance
- K3s: Cost per 1% performance drop = €0.85 (8.53/10)

For a single day of intense load testing, AKS provides better performance and stability at a higher cost. In contrast, K3s offers substantial financial savings but with a noticeable decrease in overall performance.

5.4 Operational complexity analysis

Research question 4, "What are the differences in deployment and management complexity between each platform?" is discussed in the following section. The operational complexity of deploying and managing AKS and K3s on VMSS clusters highlights significant differences in engineering time, effort and resources required. These differences are a result of different levels of abstraction in deploying the two clusters.

The deployment of AKS was streamlined and efficient, with Terraform offering options to manage almost every possible configuration in deploying a cluster with kubectl and Cilium CLI (Figure A.16) offering a straight forward process for deploying the CNI and application sets. The process was completed in 2 hours. The managed nature of AKS allowed for a significant reduction in deployment complexity with Azure handling much of the infrastructure configuration, such as virtual networks and load balancers. This required 33 lines of HCL code (Figure A.15), showing minimal operational overhead when using AKS.

Deploying K3s on VMSS required a much more involved, sophisticated approach. The deployment was created using a combination of Terraform, cloud-init and custom Bash scripts, with the total engineering time for designing, verifying and deploying the cluster was 25 hours. The infrastructure definitions for K3s required 223 lines of HCL (Figure A.17) to explicitly configure components such as virtual networks, subnets and load balancers alongside the VMSS itself. Additionally, supporting cloud-init configurations and Bash scripts required a further 185 lines of code (Figure A.18). This reflects a marked increase in the complexity of provisioning and maintaining the cluster.

In terms of flexibility, K3s on VMSS offers a higher degree of customisation and control over infrastructure and cluster configurations, along with direct access to the control plane. This comes with increased engineering effort and complexity. AKS trades some flexibility for ease of use and speed of deployment, offering a well documented and understood platform for deployments.

6 Discussion

This study evaluates the performance, cost-effectiveness and operational complexity of deploying Kubernetes clusters on AKS and K3s on VMSS. Through a combination of load testing, cost analysis and operational metrics, the findings provide insights into the trade-offs between vendor-managed and self-managed Kubernetes clusters.

The results highlight AKS' ease of deployment and robust scalability at a higher cost. In contrast, K3s delivers a more cost-effective solution but with greater operational complexity. The study also demonstrates that K3s delivered excellent resource efficiency with a smaller node footprint, especially when performing batch processing workloads. These findings underscore the importance of considering performance, operational complexity and the expected workload of the cluster when deciding on a Kubernetes platform.

6.1 Summary of key findings

This research uncovered key differences in how AKS and K3s on VMSS performed under various conditions, with insights on performance, cost-effectiveness and operational complexity. Performance was a central focus in the study, revealing that AKS maintained greater stability during simultaneous workload testing. Under heavy combined loads, AKS saw only a 1.35% drop in Online Boutique iterations and no change in TensorFlow batches completed (Figure 12), highlighting the benefit of the intense scale-up profile providing resilience in resource intensive scenarios. K3s displayed larger declines in performance in this scenario, with a 5.15% reduction in Online Boutique iterations and a 10% drop in TensorFlow batches, suggesting that either its lightweight architecture is more susceptible to resource contention in mixed-use environments or that the default scaling profile of VMSS does not pre-empt demand as effectively as AKS.

However, K3s impressed in individual testing with TensorFlow, where its smaller footprint demonstrated efficiency and outperformed AKS under the same conditions (Figure 6). This suggests that K3s could be an effective option for single application workloads or scenarios with tight resource constraints. While AKS performed strongly in individual testing with Online Boutique, it also showed a much larger deviation in iterations completed where K3s performed more consistently, albeit with fewer total iterations completed (Figure 8).

When measuring task throughput in simultaneous tests (Figure 11), K3s was more efficient when allocating CPU resources for Online Boutique compared to AKS (K3s 8.6 iterations per CPU vs AKS 7.61 iterations per CPU) but lagged behind when allocating RAM (K3s 6.45 iterations per GB RAM vs AKS 15.22 iterations per GB RAM). TensorFlow tests in this scenario exhibits K3s' performance in batch processing, out-performing AKS on efficiency in CPU allocation (K3s 2.72 batches per CPU vs AKS 1.85 batches per CPU) and RAM allocation (K3s 2.16 batches per GB RAM vs AKS 1.79 per GB RAM).

Cost-effectiveness was another factor that was explored (Subsection 5.3). Over a single day of intense simultaneous load testing, K3s was more cost-effective, with a total daily cost of the VMSS infrastructure at €8.53 compared to €13.43 for AKS. When performance degradation in these tests is taken into account, K3s had a cost-to-performance advantage, particularly for TensorFlow where on average it completed more batches than AKS despite a 10% performance drop resulting in a cost per 1% performance drop of €0.85 compared AKS's higher overall cost for maintaining stability.

Operational complexity revealed a clear contrast between the two platforms. Deploying AKS required only 33 lines of HCL code (Figure A.15) and 2 hours of total setup time to install Cilium and other applications for testing. In contrast, K3s on VMSS involved extensive engineering efforts, with 223 lines of HCL (Figure A.17) and an additional 185 lines of supporting scripts (Figure A.18), requiring a total of 25 hours to design, verify and implement.

6.2 Implication in real world application

The findings of this study have significant implications for organisations evaluating different Kubernetes platforms for their needs. Both AKS and K3s demonstrate strengths and weaknesses in simulated real-world scenarios, providing organisations with detailed information to make an informed decision based on their requirements.

For enterprises that require a robust, scalable solution that is capable of managing high-traffic applications, AKS is the clear front-runner. It exhibits stable performance under simultaneous workloads, minimal performance degradation and has a streamlined deployment process making it ideal for production environments where reliability and minimal time and effort are required.

Comparatively, K3s maintained a smaller resource footprint overall at a lower cost while still being able to operate under load. This reduced cost and resource requirement may be attractive to organisations that have a preference for efficiency and cost-effectiveness. There is a significant tradeoff in the requirement of DevOps resources and in depth knowledge on how to deploy K3s on VMSS, which would need to be considered.

Organisations with strict budget constraints may favour K3s for its lower financial costs, as demonstrated in costs both at idle and under load. K3s could also be deployed on premises at a further reduced cost and organisations who are considering a lightweight and cost effective platform for training TensorFlow models could achieve significant cost savings over vendor-managed platforms from cloud providers, without compromising performance.

AKS provides a managed experience with reduced operational complexity which is suitable for teams with limited Kubernetes experience or engineering resources and the ability to rapidly deploy a cluster makes it a natural choice for DevOps teams who manage multiple clusters or are transitioning to Kubernetes. Alternatively, K3s offers greater flexibility and customisation, allowing it to be deployed on a wide variety of hardware platforms and offers teams the opportunity to fine tune and get the most out of their resources, ideal for organisations who value customisation and control, provided they have the expertise to handle the complexity of such deployments.

6.3 Limitations of the study

While this study provides valuable insights into the performance, cost and operational complexity of AKS and K3s on VMSS, there are several limitations that should be addressed. These limitations suggest that while the study provides valuable insights, further research featuring a wider range of workloads, customisations, longer testing periods and more varied testing conditions would all provide further insights into these platforms.

6.3.1 Limited scope of testing

The analysis focuses on testing with two applications under specific conditions. These workloads, while representative of real-world workloads, do not cover the full spectrum of applications that may run on Kubernetes in production environments. Therefore, the conclusions drawn can not be generalised to cover other types of applications or use cases.

6.3.2 Limited testing conditions

The study assessed the performance of both clusters under specific load conditions, in individual testing and simultaneous application testing. More diverse testing conditions, such as varying traffic patterns, TensorFlow workloads and fault tolerance testing could provide a more comprehensive understanding of each platform's strengths and weaknesses in different scenarios.

6.3.3 Infrastructure variability

The deployment configurations for both clusters were not identical, with K3s on VMSS requiring extensive configuration scripts. This introduced potential variables in the deployment process, including the requirement of each VMSS node that spawned to run configuration scripts on first boot to download and install the required software to join the K3s cluster. This added operational complexity and may have been a significant disadvantage in comparing node scaling performance.

6.3.4 Limited performance metrics

The study analysed performance based on metrics such as pod scaling, resource usage and application performance, there were other aspects of application performance that were not measured, latency and response time, for example. A wider set of performance metrics could provide a more nuanced understanding of each platform's performance.

6.3.5 User specific customisations

Both platforms offer a wide range of options in terms of configuration and optimisation. This study used default configurations for both platforms, which may not represent best practice or optimisations that could be used in production environments. Differences in user specific configurations could result in different performance and cost results.

7 Conclusion

The findings of this study demonstrate that both AKS and K3s have distinct advantages and limitations when deployed on similar Azure based environments. AKS features a streamlined setup process with automated management of many deployment components and showed greater stability under intense load. However, this came at a financial cost and the reliance on Azure native integrations may make it less appealing for cost-sensitive deployments or scenarios that require a high level of customisation.

K3s, in comparison, operated with a smaller footprint overall and performed strongly in TensorFlow tasks at a significantly lower cost than AKS. The increased engineering effort and complexity of deployment make this a less suitable option for organisations who have a preference for rapid scalability without deep technical knowledge.

7.1 Future work

This research provides a starting point for further exploration into Kubernetes distributions operating in non edge-computing environments. Future studies could compare these platforms on a static node count to compare performance in a more direct manner. Another area of interest would be to deploy both platforms with the goal of optimising their configurations, potentially finding greater performance gains.

Similarly, a larger comparison with more Kubernetes distributions like MiniKube, OpenShift and MicroK8s would provide a wider perspective on container orchestration platforms. Finally, exploring more applications and workloads would yield deeper insights into the performance of Kubernetes distributions across a larger variety of workloads and scenarios.

A Appendices

A.1 Detailed testing configurations

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3 from prometheus_client import start_http_server, Gauge
4 import time
5
6 # Start Prometheus
7 start_http_server(8000)
8
9 # Prometheus metrics
10 accuracy_gauge = Gauge('tensorflow', 'Accuracy of the tensorflow model')
11 loss_gauge = Gauge('tensorflow', 'Loss of the tensorflow model')
12 batches_processed_gauge = Gauge('tensorflow_batches_processed', 'Number of training batches processed')
13 epochs_completed_gauge = Gauge('tensorflow_epochs_completed', 'Number of training epochs completed')
14
15 try:
16     import tensorflow_datasets as tfds
17 except ImportError:
18     import os
19     os.system('pip install tensorflow-datasets')
20     import tensorflow_datasets as tfds
21
22 # Preprocess
23 def preprocess(image, label):
24     image = tf.image.resize(image, (224, 224))
25     image = tf.keras.applications.mobilenet_v2.preprocess_input(image)
26     return image, label
27
28 # Load the dataset
29 (train_ds, val_ds), ds_info = tfds.load(
30     'cifar10',
31     split=['train', 'test'],
32     as_supervised=True,
33     with_info=True,
34     data_dir='/tmp/tfds'
35 )
36
37 # Smaller subset and bigger batch
38 train_ds = train_ds.take(1000).map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)
39 val_ds = val_ds.take(200).map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)
40
41 # Pre-trained model
42 base_model = tf.keras.applications.MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
43 base_model.trainable = False
44
45 # Add custom classification layers with fewer units
46 num_classes = ds_info.features['label'].num_classes
47 model = models.Sequential([
48     base_model,
49     layers.GlobalAveragePooling2D(),
50     layers.Dense(128, activation='relu'), # Reduced units
51     layers.Dropout(0.5),
52     layers.Dense(num_classes, activation='softmax')
53 ])
54
55 # Compile model
56 model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Figure A.13: Python code for configuring TensorFlow training model

```
57 # Training loop
58 total_batches = 1000 // 32 # Total number of batches (1000 samples, batch size of 32)
59 for epoch in range(2): # Reduce epochs for faster completion
60     history = model.fit(train_ds, validation_data=val_ds, epochs=1)
61
62     # Metrics accuracy and loss
63     accuracy = history.history['accuracy'][0]
64     loss = history.history['loss'][0]
65     accuracy_gauge.set(accuracy)
66     loss_gauge.set(loss)
67
68     # Update metrics batches
69     batches_processed_gauge.inc(total_batches)
70     epochs_completed_gauge.inc(1)
71
72     # Sleep to stop endpoint from overloading
73     time.sleep(1)
```

Figure A.13: Python code for configuring TensorFlow training model

```

1  import http from 'k6/http';
2  import { check, sleep } from 'k6';
3  // Product IDs
4  const productIds = [
5      '66VCHSJNUP',
6      '0LJCESPC7Z',
7      '1YMWWN1N40',
8      'L9ECAV7KIM',
9      '2ZYFJ3GM2N',
10     'OPUK6V6EV0',
11     'LS4PSXUNUM',
12     '9SIQT8TOJO',
13     '6E92ZMYFFZ',
14 ];
15
16 export const options = {
17     vus: 1000,
18     duration: '30m',
19 };
20 export default function () {
21     // Randomly select a product ID from the list
22     const randomProductId = productIds[Math.floor(Math.random() * productIds.length)];
23     const productUrl = `https://{url}/product/${randomProductId}`;
24     // Open product page
25     const productPageRes = http.get(productUrl);
26     check(productPageRes, {
27         'Product page loaded': (r) => r.status === 200,
28     });
29     // Add to cart
30     const cartUrl = 'https://{url}/cart';
31     const cartRes = http.get(cartUrl);
32     check(cartRes, {
33         'Cart page loaded': (r) => r.status === 200,
34     });
35     // Go to checkout
36     const checkoutUrl = 'https://{url}/cart/checkout';
37     const formData = {
38         email: "someone@example.com",
39         street_address: "1600 Amphitheatre Parkway",
40         zip_code: "94043",
41         city: "Mountain View",
42         state: "CA",
43         country: "United States",
44         credit_card_number: "4432801561520454",
45         credit_card_expiration_month: "5",
46         credit_card_expiration_year: "2025",
47         credit_card_cvv: "672",
48     };
49
50     const headers = {
51         'Content-Type': 'application/x-www-form-urlencoded',
52     };
53     const checkoutRes = http.post(checkoutUrl, formData, { headers });
54     // Confirm checkout
55     check(checkoutRes, {
56         'Order placed': (r) => r.status === 302 || r.status === 200, // Check for redirect or success
57     });
58     // Complete order
59     const isSuccess = checkoutRes.body.includes("Your order is complete!");
60
61     check(checkoutRes, {
62         'Checkout successful': () => isSuccess,
63     });
64     sleep(1);
65 }

```

Figure A.14: K6 test for Online Boutique

A.2 Example code for environment setup

```
1 resource "azurerm_kubernetes_cluster" "aks" {
2   name           = "aks-cluster"
3   location       = var.location
4   resource_group_name = var.rg
5   dns_prefix     = "aks-cluster"
6   kubernetes_version = "1.28.5"
7   node_resource_group = "${var.rg}-aks"
8
9   default_node_pool {
10    min_count = 1
11    max_count = 10
12    name      = "nodepool"
13    vm_size   = "standard_d2as_v5"
14    os_disk_type = "Managed"
15    os_disk_size_gb = 30
16    type        = "VirtualMachineScaleSets"
17    zones       = ["1", "2", "3"]
18    auto_scaling_enabled = true
19  }
20
21  identity {
22    type = "SystemAssigned"
23  }
24
25  http_application_routing_enabled = true
26
27  lifecycle {
28    ignore_changes = [default_node_pool[0].node_count]
29  }
30  network_profile {
31    network_plugin = "none"
32  }
33 }
```

Figure A.15: Terraform code to deploy AKS cluster

```
1 cilium install --version 1.16.1 \
2   --set ipam.operator.clusterPoolIPv4PodCIDRList="10.42.0.0/16" \
3   --set k8sServiceHost="10.0.1.10" \
4   --set k8sServicePort="6443" \
5   --set kubeProxyReplacement=true \
6   --set hubble.enabled="true" \
7   --set hubble.ui.enabled="true" \
8   --set hubble.relay.enabled="true" \
9   --set envoy.securityContext.capabilities.keepCapNetBindService=true \
10  --set gatewayAPI.enabled=true \
11  --set gatewayAPI.loadbalancerMode=shared \
12  --set serviceMesh.enabled=true \
13  --set l2announcements.enabled=true
```

Figure A.16: Command line to install Cilium on AKS

```

1  # Virtual Network
2  resource "azurerm_virtual_network" "k3s_vnet" {
3      name                = "k3sVNet"
4      address_space       = ["10.0.0.0/16"]
5      location            = var.location
6      resource_group_name = var.rg
7  }
8
9  # Subnet
10 resource "azurerm_subnet" "k3s_subnet" {
11     name                = "k3sSubnet"
12     resource_group_name = var.rg
13     virtual_network_name = azurerm_virtual_network.k3s_vnet.name
14     address_prefixes     = ["10.0.1.0/24"]
15 }
16
17 # Associate the NSG
18 resource "azurerm_subnet_network_security_group_association" "k3s_nsg_association" {
19     subnet_id = azurerm_subnet.k3s_subnet.id
20     network_security_group_id = azurerm_network_security_group.k3s_nsg.id
21 }
22
23 # Network Security Group
24 resource "azurerm_network_security_group" "k3s_nsg" {
25     name                = "k3sNSG"
26     location            = var.location
27     resource_group_name = var.rg
28
29     security_rule {
30         name                = "AllowSSH"
31         priority            = 100
32         direction           = "Inbound"
33         access              = "Allow"
34         protocol            = "Tcp"
35         source_port_range   = "*"
36         destination_port_range = "22"
37         source_address_prefix = "*"
38         destination_address_prefix = "*"
39     }
40
41     security_rule {
42         name                = "AllowHTTPS"
43         priority            = 120
44         direction           = "Inbound"
45         access              = "Allow"
46         protocol            = "Tcp"
47         source_port_range   = "*"
48         destination_port_range = "443"
49         source_address_prefix = "*"
50         destination_address_prefix = "*"
51     }
52
53     security_rule {
54         name                = "AllowK3s"
55         priority            = 130
56         direction           = "Inbound"
57         access              = "Allow"
58         protocol            = "Tcp"
59         source_port_range   = "*"
60         destination_port_ranges = ["6443", "10250", "2379-2380", "30000-32767"]
61         source_address_prefix = "*"
62         destination_address_prefix = "*"
63     }
64 }

```

Figure A.17: Terraform code for deploying VMSS

```

1  # Public IP for Load Balancer
2  resource "azurerm_public_ip" "k3s_lb_public_ip" {
3      name                = "k3sPublicIP"
4      location            = var.location
5      resource_group_name = var.rg
6      allocation_method   = "Static"
7      sku                 = "Standard"
8  }
9
10 # Load Balancer
11 resource "azurerm_lb" "k3s_lb" {
12     name                = "k3sLoadBalancer"
13     location            = var.location
14     resource_group_name = var.rg
15     sku                 = "Standard"
16
17     frontend_ip_configuration {
18         name                = "k3sFrontEnd"
19         public_ip_address_id = azurerm_public_ip.k3s_lb_public_ip.id
20     }
21 }
22
23 # Load Balancer Backend Pool
24 resource "azurerm_lb_backend_address_pool" "k3s_backend_pool" {
25     name                = "k3sBackendPool"
26     loadbalancer_id     = azurerm_lb.k3s_lb.id
27 }
28
29 # Assign backend to address to primary node
30 resource "azurerm_lb_backend_address_pool_address" "primary" {
31     name                = "primary"
32     backend_address_pool_id = azurerm_lb_backend_address_pool.k3s_backend_pool.id
33     ip_address          = "10.0.1.4"
34     virtual_network_id   = azurerm_virtual_network.k3s_vnet.id
35 }
36
37 # Load Balancer Probe
38 resource "azurerm_lb_probe" "k3s_http_probe" {
39     name                = "k3sHttpProbe"
40     loadbalancer_id     = azurerm_lb.k3s_lb.id
41     protocol            = "Http"
42     port                = 443
43     request_path        = "/healthz"
44     interval_in_seconds = 5
45     number_of_probes    = 2
46 }
47
48 resource "azurerm_lb_probe" "k3s_ssh_probe" {
49     name                = "k3sSSHProbe"
50     loadbalancer_id     = azurerm_lb.k3s_lb.id
51     protocol            = "Tcp"
52     port                = 22
53     interval_in_seconds = 5
54     number_of_probes    = 2
55 }
56
57 resource "azurerm_lb_probe" "k3s_k8s_probe" {
58     name                = "k3sk8sProbe"
59     loadbalancer_id     = azurerm_lb.k3s_lb.id
60     protocol            = "Tcp"
61     port                = 6443
62     interval_in_seconds = 5
63     number_of_probes    = 2
64 }

```

Figure A.17: Terraform code for deploying VMSS

```

1  # Load Balancer Rule
2  resource "azurerm_lb_rule" "k3s_lb_rule" {
3      name                        = "k3sHTTPRule"
4      loadbalancer_id            = azurerm_lb.k3s_lb.id
5      protocol                   = "Tcp"
6      frontend_port              = 443
7      backend_port               = 443
8      frontend_ip_configuration_name = "k3sFrontEnd"
9      backend_address_pool_ids    = [azurerm_lb_backend_address_pool.k3s_backend_pool.id]
10     probe_id                   = azurerm_lb_probe.k3s_http_probe.id
11     enable_floating_ip = false
12     idle_timeout_in_minutes = 4
13     load_distribution = "Default"
14 }
15
16 resource "azurerm_lb_rule" "k3s_lb_ssh_rule" {
17     name                        = "k3sSSHRule"
18     loadbalancer_id            = azurerm_lb.k3s_lb.id
19     protocol                   = "Tcp"
20     frontend_port              = 22
21     backend_port               = 22
22     frontend_ip_configuration_name = "k3sFrontEnd"
23     backend_address_pool_ids    = [azurerm_lb_backend_address_pool.k3s_backend_pool.id]
24     probe_id                   = azurerm_lb_probe.k3s_ssh_probe.id
25     enable_floating_ip = false
26     idle_timeout_in_minutes = 4
27     load_distribution = "Default"
28 }
29
30 resource "azurerm_lb_rule" "k3s_lb_k8s_rule" {
31     name                        = "k3sk8sRule"
32     loadbalancer_id            = azurerm_lb.k3s_lb.id
33     protocol                   = "Tcp"
34     frontend_port              = 6443
35     backend_port               = 6443
36     frontend_ip_configuration_name = "k3sFrontEnd"
37     backend_address_pool_ids    = [azurerm_lb_backend_address_pool.k3s_backend_pool.id]
38     probe_id                   = azurerm_lb_probe.k3s_ssh_probe.id
39     enable_floating_ip = false
40     idle_timeout_in_minutes = 4
41     load_distribution = "Default"
42 }

```

Figure A.17: Terraform code for deploying VMSS

```

1  # Virtual Machine Scale Set
2  resource "azurerm_linux_virtual_machine_scale_set" "k3s_vmss" {
3      name                = "k3sVMSS"
4      location            = var.location
5      resource_group_name = var.rg
6      sku                 = "standard_d2as_v5"
7      instances           = 2
8      admin_username      = "adminuser"
9
10     admin_ssh_key {
11         username   = "adminuser"
12         public_key = local.first_public_key
13     }
14
15     source_image_reference {
16         publisher = "Canonical"
17         offer     = "0001-com-ubuntu-server-jammy"
18         sku       = "22_04-lts"
19         version   = "latest"
20     }
21
22     os_disk {
23         caching              = "ReadWrite"
24         storage_account_type = "Standard_LRS"
25     }
26
27     network_interface {
28         name    = "k3sNetProfile"
29         primary = true
30
31         ip_configuration {
32             name       = "k3sIPConfig"
33             subnet_id  = azurerm_subnet.k3s_subnet.id
34             primary    = true
35         }
36     }
37     custom_data = local.install_k3s_control
38     tags = {
39         "role" = "primary"
40     }
41 }
42
43 output "vmss_id" {
44     value = azurerm_linux_virtual_machine_scale_set.k3s_vmss.id
45 }
46
47 locals {
48     first_public_key = file("~/ssh/id_rsa_azure.pub")
49     install_k3s_control = filebase64("cloud_init.yaml")
50 }

```

Figure A.17: Terraform code for deploying VMSS

```

1  #!/bin/bash
2  set -x
3  if [ -f /etc/k3s-install-done ]; then
4      echo "K3s installation already completed. Exiting."
5      exit 0
6  fi
7  # Env Vars
8  STORAGE_ACCOUNT_NAME=""
9  CONTAINER_NAME="k3s-keys"
10 BLOB_FILE="node-token"
11 LOCAL_TOKEN_PATH="/var/lib/rancher/k3s/server/node-token"
12 LOCAL_TOKEN_TEMP="/tmp/node-token"
13 AZURE_STORAGE_ACCESS_KEY=""
14
15 # Install Azure CLI
16 if ! command -v az &> /dev/null
17 then
18     curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
19 fi
20
21 # Auth with Azure storage
22 az storage account keys list --account-name $STORAGE_ACCOUNT_NAME --output table
23
24 # Update and install dependencies
25 sudo apt-get update -y
26 sudo apt-get install -y curl jq etcd
27 sudo systemctl enable etcd
28 sudo systemctl start etcd
29 # Upload node-token to Azure Blob storage
30 upload_token() {
31     az storage blob upload --account-name $STORAGE_ACCOUNT_NAME --account-key $AZURE_STORAGE_ACCESS_KEY \
32     --container-name $CONTAINER_NAME --file $LOCAL_TOKEN_PATH --name $BLOB_FILE --overwrite
33 }
34 # Download node-token from Azure Blob storage
35 download_token() {
36     az storage blob download --account-name $STORAGE_ACCOUNT_NAME --account-key $AZURE_STORAGE_ACCESS_KEY \
37     --container-name $CONTAINER_NAME --name $BLOB_FILE --file $LOCAL_TOKEN_TEMP --overwrite
38 }
39 # Upload KUBECONFIG
40 upload_kube() {
41     LOCAL_FILE_PATH="/etc/rancher/k3s/k3s.yaml"
42     BLOB_FILE="k3s.yaml"
43
44     az storage blob upload \
45     --account-name $STORAGE_ACCOUNT_NAME \
46     --account-key $AZURE_STORAGE_ACCESS_KEY \
47     --container-name $CONTAINER_NAME \
48     --file $LOCAL_FILE_PATH \
49     --name $BLOB_FILE \
50     --overwrite
51 }

```

Figure A.18: Bash script for deploying K3s on VMSS

```

1  # K3s install control plane
2  install_k3s_control() {
3      export INSTALL_K3S_VERSION="v1.28.5+k3s1"
4      export K3S_KUBECONFIG_MODE="644"
5      export INSTALL_K3S_EXEC="server --disable=traefik --flannel-backend none --disable-kube-proxy \
6      --disable=service-lb --disable-network-policy --datastore-endpoint=etcd \
7      --cluster-cidr=10.42.0.0/16 --service-cidr=10.43.0.0/16 --node-ip=10.0.1.10"
8
9      # Run K3s install
10     curl -sL https://get.k3s.io | sh -
11 }
12
13 # K3s install on worker
14 install_k3s_worker() {
15     export INSTALL_K3S_VERSION="v1.28.5+k3s1"
16     export K3S_KUBECONFIG_MODE="644"
17     export K3S_URL="https://10.0.1.10:6443"
18     export K3S_TOKEN=$(cat $LOCAL_TOKEN_TEMP)
19
20     # Run K3s install
21     curl -sL https://get.k3s.io | sh -
22 }
23
24 # Wait for primary
25 wait_for_k3s_api() {
26     local max_attempts=30
27     local attempt=0
28     local delay=10
29
30     while [[ $attempt -lt $max_attempts ]]; do
31         if curl -k https://10.0.1.10:6443/version; then
32             echo "K3s API is ready."
33             return 0
34         else
35             echo "Waiting for K3s API to be ready... Attempt $((attempt + 1))/$max_attempts"
36             sleep $delay
37             attempt=$((attempt + 1))
38         fi
39     done
40
41     echo "K3s API did not become ready in time."
42     return 1
43 }

```

Figure A.18: Bash script for deploying K3s on VMSS

```

1  # Check if the node is primary or worker node
2  if [[ "$(hostname)" == "k3sVMSS000000" ]]; then
3      echo "This is the control node. Installing K3s control plane..."
4      install_k3s_control
5      upload_token
6
7      # Install Gateway API deps
8      export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
9      kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/gateway-api/v1.1.0/config/crd/standard/gateway.networking.k8s.io_gatewayclasses.yaml
10     kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/gateway-api/v1.1.0/config/crd/standard/gateway.networking.k8s.io_gateways.yaml
11     kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/gateway-api/v1.1.0/config/crd/standard/gateway.networking.k8s.io_httproutes.yaml
12     kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/gateway-api/v1.1.0/config/crd/standard/gateway.networking.k8s.io_referencegrants.yaml
13     kubectl apply -f https://raw.githubusercontent.com/kubernetes-sigs/gateway-api/v1.1.0/config/crd/standard/gateway.networking.k8s.io_grpcroutes.yaml
14
15     # Install Helm
16     curl https://baltocdn.com/helm/signing.asc | gpg --dearmor | sudo tee /usr/share/keyrings/helm.gpg > /dev/null
17     sudo apt-get install apt-transport-https --yes
18     echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/helm.gpg] https://baltocdn.com/helm/stable/debian/ all main"
19     / | sudo tee /etc/apt/sources.list.d/helm-stable-debian.list
20     sudo apt-get update
21     sudo apt-get install -y helm
22
23     # Install Cilium CLI
24     CILIUM_CLI_VERSION=$(curl -s https://raw.githubusercontent.com/cilium/cilium-cli/main/stable.txt)
25     curl -L --fail --remote-name-all https://github.com/cilium/cilium-cli/releases/download/${CILIUM_CLI_VERSION}/cilium-linux-amd64.tar.gz
26     sudo tar xzvf cilium-linux-amd64.tar.gz -C /usr/local/bin
27
28     # Install Cilium on both control and worker nodes
29     cilium install --version 1.16.1 --set=ipam.operator.clusterPoolIPv4PodCIDRList="10.42.0.0/16" --set=k8sServiceHost="10.0.1.10" \
30     --set=k8sServicePort="6443" --set kubeProxyReplacement=true --set=hubble.enabled=true --set=hubble.ui.enabled=true \
31     --set=hubble.relay.enabled=true --set envoy.securityContext.capabilities.keepCapNetBindService=true \
32     --set gatewayAPI.enabled=true --set gatewayAPI.loadbalancerMode=shared --set serviceMesh.enabled=true \
33     --set l2announcements.enabled=true
34
35     # Upload kubeconfig
36     upload_kube
37
38     # Create config.yaml
39     touch /etc/rancher/k3s/config.yaml
40     sudo sh -c 'echo "tls-san:\n  - \"10.0.1.10\"\n  - \"52.138.215.91\" \">> /etc/rancher/k3s/config.yaml'
41
42 else
43     echo "This is a worker node. Installing K3s worker node..."
44     # Wait for K3s API to be ready
45     wait_for_k3s_api
46     # Download node token from Azure Blob storage
47     sleep 30s
48     download_token
49     install_k3s_worker
50 fi
51
52 # Restart K3s to apply changes (control plane only)
53 if [[ "$(hostname)" == "k3sVMSS000000" ]]; then
54     sudo systemctl restart k3s
55 fi

```

Figure A.18: Bash script for deploying K3s on VMSS

References

- Bahy, Muhammad Bintang et al. (2023). ‘Resource Utilization Comparison of KubeEdge, K3s, and Nomad for Edge Computing’. In: *2023 10th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI)*. IEEE, pp. 321–327.
- Bohm, Sebastian and Guido Wirtz (2021). ‘Profiling Lightweight Container Platforms: MicroK8s and K3s in Comparison to Kubernetes.’ In: *ZEUS*, pp. 65–73.
- Bryant, Lincoln et al. (2024). ‘Kubernetes Deployment Options for On-Prem Clusters’. In: *arXiv preprint arXiv:2407.01620*.
- Chen Keyu, Bi et al. (2024). ‘Deep learning and machine learning, advancing big data analytics and management: Tensorflow pretrained models’. In: *arXiv preprint arXiv:2409.13566*.
- Ćłapa, Konrad, Krzysztof Grudzień and Artur Sierszeń (2023). ‘Performance Analysis of Machine Learning Platforms Using Cloud Native Technology on Edge Devices’. In: *Wojciechowski A.(Ed.), Lipiński P.(Ed.), Progress in Polish Artificial Intelligence Research 4, Seria: Monografie Politechniki Łódzkiej Nr. 2437, Wydawnictwo Politechniki Łódzkiej, Łódź 2023, ISBN 978-83-66741-92-8, doi: 10.34658/9788366741928*.
- CNCF (Apr. 2024). *CNCF Annual Survey 2023*. CNCF. URL: <https://www.cncf.io/reports/cncf-annual-survey-2023/#findings>.
- Darbinyan, Davit (2024). ‘Kubeless: A Novel Architecture for Kubernetes’ Control Plane’. PhD thesis. Vrije Universiteit Amsterdam.
- Ernerstedt, Jonas (2023). *An evaluation of tools for verifying non-functional requirements for cloud deployed applications*.
- Farrel, Gabrielle Evan et al. (2023). ‘Scalable Edge Computing Cluster Using a Set of Raspberry Pi: A Framework’. In: *Proceedings of the 8th International Conference on Sustainable Information Engineering and Technology*, pp. 287–296.
- Fogli, Mattia et al. (2021). ‘Performance evaluation of kubernetes distributions (k8s, k3s, kubeedge) in an adaptive and federated cloud infrastructure for dis-

- advantaged tactical networks’. In: *2021 International Conference on Military Communication and Information Systems (ICMCIS)*. IEEE, pp. 1–7.
- Franssens, Nills, Shivakumar Gopalakrishnan and Gunther Lenz (2021). *Hands-on Kubernetes on Azure: Use Azure Kubernetes Service to automate management, scaling, and deployment of containerized applications*. Packt Publishing Ltd.
- Google (2024). *Googlecloudplatform/microservices-demo: Sample Cloud-first application with 10 microservices showcasing Kubernetes, Istio, and grpc*. Accessed: 2024-11-06. GitHub. URL: <https://github.com/GoogleCloudPlatform/microservices-demo>.
- Gurin, Dmytro et al. (2024). ‘MobileNetv2 Neural Network Model for Human Recognition and Identification in the Working Area of a Collaborative Robot’. In: *Multidisciplinary Journal of Science and Technology* 4.8, pp. 5–12.
- Heckmann, Owen and Arun Ravindran (2023). ‘Evaluating Kubernetes at the Edge for Fault Tolerant Multi-Camera Computer Vision Applications’. In: *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)*. IEEE, pp. 269–271.
- Holopainen, Matti (2021). ‘Monitoring Container Environment with Prometheus and Grafana’. In.
- Karakacs, Buket et al. (2023). ‘Enhancing Security in Communication Applications Deployed on Kubernetes: Best Practices and Service Mesh Analysis’. MA thesis.
- Koukis, Georgios et al. (2024). ‘An Open-Source Experimentation Framework for the Edge Cloud Continuum’. In: *arXiv preprint arXiv:2403.10977*.
- Koziolek, Heiko and Nafise Eskandani (2023). ‘Lightweight kubernetes distributions: A performance comparison of microk8s, k3s, k0s, and microshift’. In: *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*, pp. 17–29.
- Kuikka, Sami (2024). ‘Kubernetes Networking: Comparative Insights into API Gateways and Service Mesh Implementations’. In.
- Kumar, E Saravana et al. (2024). ‘Comparative Study and Analysis of Cloud Container Technology’. In: *2024 11th International Conference on Computing for Sustainable Global Development (INDIACom)*. IEEE, pp. 1681–1686.

- Lawrence, Jackson (2024). ‘Characterizing a Web Server Deployed in a Lightweight Kubernetes Cluster in an IoT Computing Environment’. MA thesis. The University of Alabama in Huntsville.
- Li, Hao, Gopi Krishnan Rajbahadur and Cor-Paul Bezemer (2024). ‘Studying the impact of tensorflow and pytorch bindings on machine learning software quality’. In: *ACM Transactions on Software Engineering and Methodology*.
- Mehdi, Abbas and Ranjan Walia (2023). ‘Terraform: Streamlining Infrastructure Deployment and Management Through Infrastructure as Code’. In: *2023 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*. IEEE, pp. 851–856.
- Mfula, Harrison, Antti Ylä-Jääski and Jukka K Nurminen (2021). ‘Seamless kubernetes cluster management in multi-cloud and edge 5G applications’. In: *International Conference on High Performance Computing & Simulation*.
- Mondal, Subrota Kumar, Zhen Zheng and Yuning Cheng (2024). ‘On the Optimization of Kubernetes toward the Enhancement of Cloud Computing’. In: *Mathematics* 12.16, p. 2476.
- Mušić, Din, Jernej Hribar and Carolina Fortuna (2024). ‘Digital transformation with a lightweight on-premise PaaS’. In: *Future Generation Computer Systems*.
- Nagy, EnikHo et al. (2021). ‘Cloud-agnostic architectures for machine learning based on Apache Spark’. In: *Advances in Engineering Software* 159, p. 103029.
- Nelli, Fabio (2023). ‘Deep Learning with TensorFlow’. In: *Python Data Analytics: With Pandas, NumPy, and Matplotlib*. Springer, pp. 289–321.
- Normelli, Gustav (2024). *Benchmarking Message Brokers on Kubernetes: A Case Study in how to Evaluate Inter-Service Communication Technology*.
- Paloviita, Olli (2022). ‘Infrastructure as Code for Managed Service Providers: A Case Study’. In.
- Perez, Ricardo et al. (2023). ‘Moving microgrid hierarchical control to an SDN-based kubernetes cluster: a framework for reliable and flexible energy distribution’. In: *Sensors* 23.7, p. 3395.
- Pivotto, Julien and Brian Brazil (2023). *Prometheus: Up & Running*. " O'Reilly Media, Inc."

- Schumann, Maximilian (2024). ‘Conceptual design of a container-based system landscape orchestrated by Kubernetes’. B.S. thesis.
- Sharma, Anurag (2023). ‘Evaluate Kubernetes for Stateful and highly available enterprise database solutions’. MA thesis. Oslomet-storbyuniversitetet.
- Sorensen, Robin (2023). *An evaluation of edge deployment models for Kubernetes*.
- Telenyk, Sergii et al. (2021). ‘A comparison of kubernetes and kubernetes-compatible platforms’. In: *2021 11th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (ID-AACS)*. Vol. 1. IEEE, pp. 313–317.
- Todorov, Milen Hrabarov (2022). ‘Deploying Different Lightweight Kubernetes on Raspberry Pi Cluster’. In: *2022 30th National Conference with International Participation (TELECOM)*. IEEE, pp. 1–4.
- Turin, Gianluca et al. (2023). ‘Predicting resource consumption of Kubernetes container systems using resource models’. In: *Journal of Systems and Software* 203, p. 111750.
- Zavarella, Timothy D (2022). ‘A methodology for using eBPF to efficiently monitor network behavior in Linux Kubernetes clusters’. PhD thesis. Massachusetts Institute of Technology.
- Zerouali, Ahmed, Ruben Opdebeeck and Coen De Roover (2023). ‘Helm charts for Kubernetes applications: Evolution, outdatedness and security risks’. In: *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, pp. 523–533.